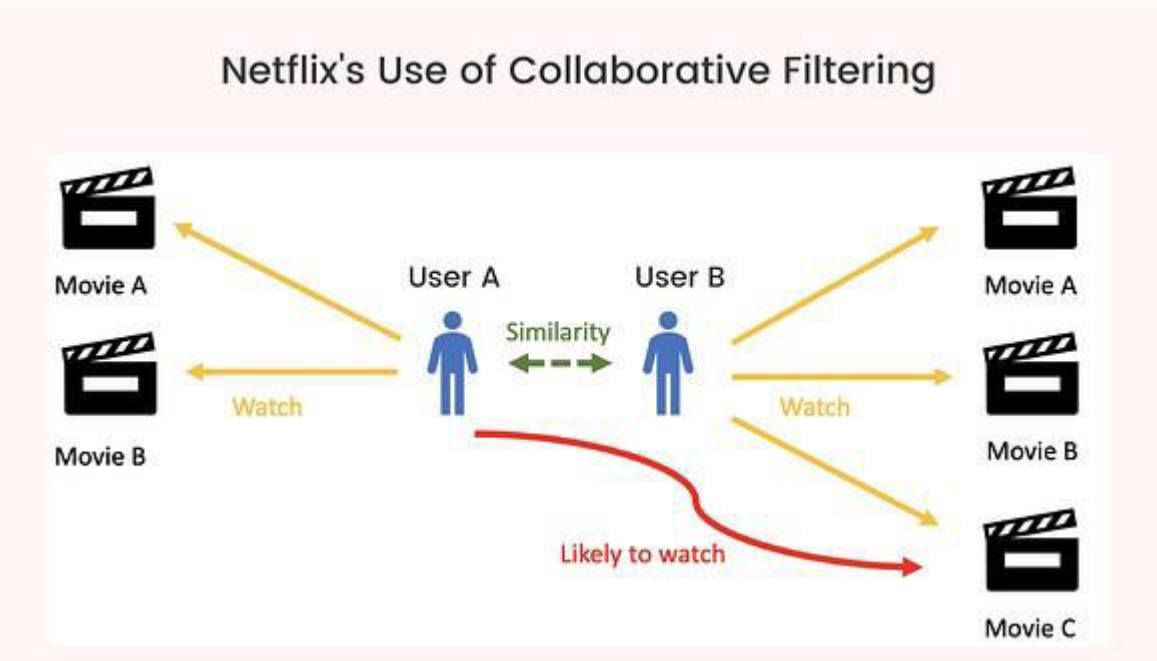
Collaborative Filtering

Collaborative filtering is a recommendation technique that predicts user preferences by leveraging historical interactions. It is widely used in recommendation systems such as Netflix, Amazon, and Spotify to suggest items based on user behaviour.



1. User-Based Collaborative Filtering

User-based collaborative filtering identifies users with similar preferences and recommends items that similar users have liked. It operates in the following steps:

1. **Identify Similar Users** – Compute similarity between users based on their past interactions using metrics such as:
    - Cosine similarity
    - Pearson correlation
2. **Find Relevant Items** – Identify items that similar users have interacted with but the target user has not.

3. **Generate Recommendations** – Rank and suggest items based on similarity scores.

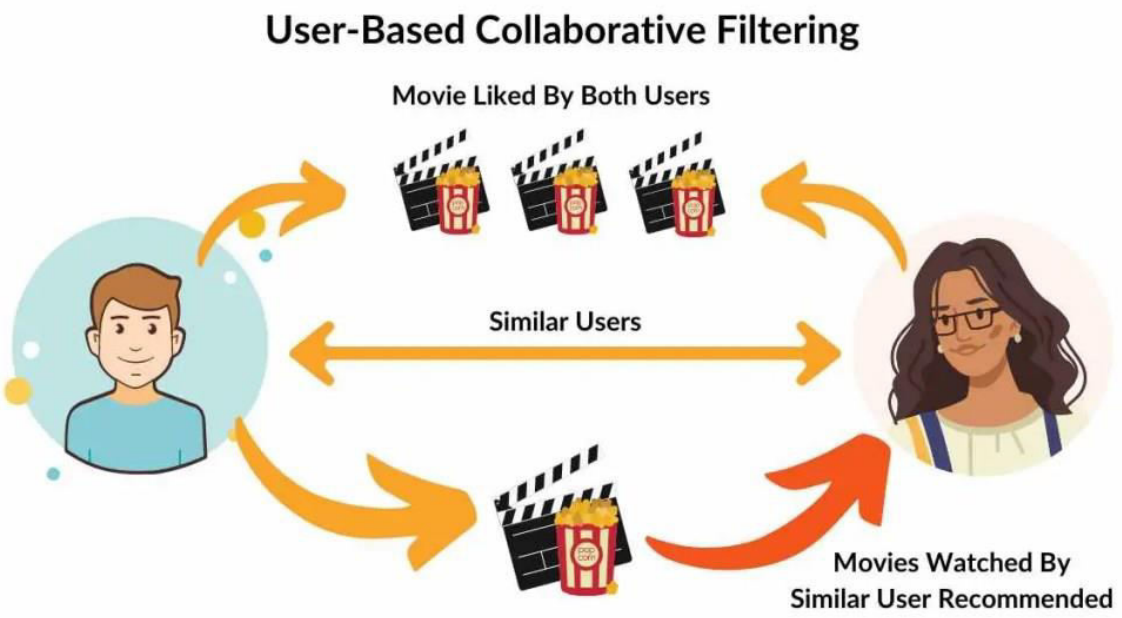
Example: If User A and User B have similar movie-watching histories, and User B watches a new movie, the system may recommend that movie to User A.

Applications of Collaborative Filtering

- Movie and TV show recommendations (Netflix, Hulu)
- E-commerce product suggestions (Amazon, eBay)
- Music and podcast recommendations (Spotify, Apple Music)
- Online learning course recommendations (Coursera, Udemy)

Challenges of Collaborative Filtering

1. **Cold Start Problem** – New users or items have little interaction data.
2. **Sparsity** – User-item interaction matrices are often sparse.
3. **Scalability** – Large-scale data requires efficient computation.
4. **Diversity** – Over-recommendation of popular items can reduce diversity.




```python
# Install the required library if not installed
# !pip install scikit-surprise

from surprise import Dataset, Reader, KNNBasic
from surprise.model_selection import train_test_split
from surprise import accuracy
import pandas as pd

# Load built-in MovieLens dataset (100k ratings)
data = Dataset.load_builtin('ml-100k')

# Split dataset into training and testing sets
trainset, testset = train_test_split(data, test_size=0.2)

# Define User-Based Collaborative Filtering model
sim_options = {
    'name': 'cosine',  # Use cosine similarity
    'user_based': True  # Enable user-based filtering
}

# Use KNNBasic for user-based filtering
user_cf_model = KNNBasic(sim_options=sim_options)

# Train the model
user_cf_model.fit(trainset)

# Make predictions
predictions = user_cf_model.test(testset)
```

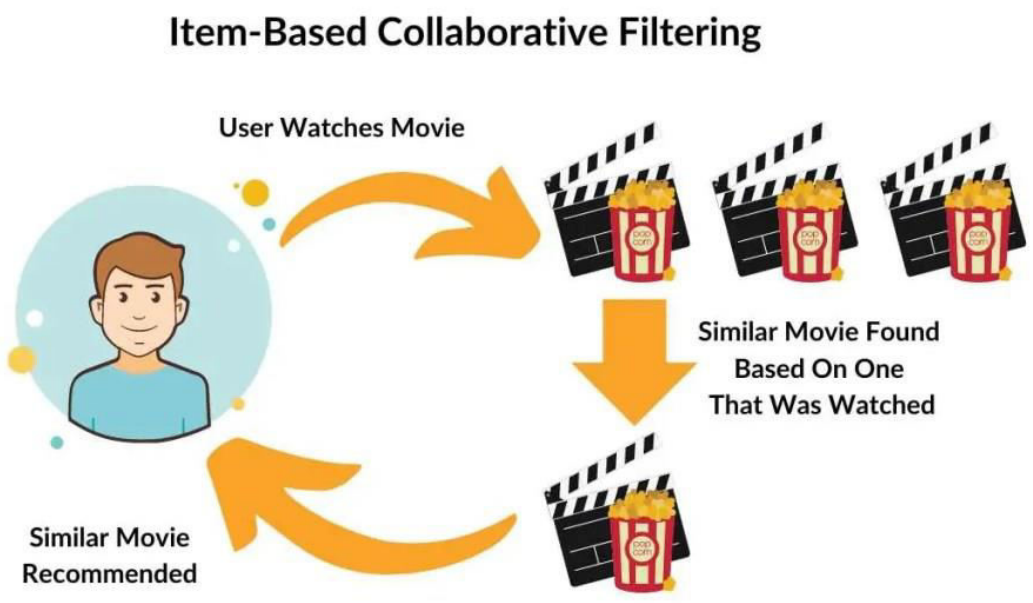
```python
# Make predictions
predictions = user_cf_model.test(testset)

# Evaluate using RMSE (Root Mean Squared Error)
rmse = accuracy.rmse(predictions)
print(f"User-Based CF Model RMSE: {rmse}")

# Example: Predict rating for a specific user-item pair (user 1, item 50)
pred = user_cf_model.predict(uid=1, iid=50)
print(f"Predicted rating for user 1 on item 50: {pred.est}")
```

2. Item-Based Collaborative Filtering

Item-based collaborative filtering recommends items similar to those a user has interacted with. Instead of comparing users, it compares items using similarity measures such as:

1. **Compute Item Similarity** – Determine how similar two items are based on user interactions (e.g., people who liked item X also liked item Y).
2. **Generate Recommendations** – Suggest items that are similar to those a user has already engaged with.

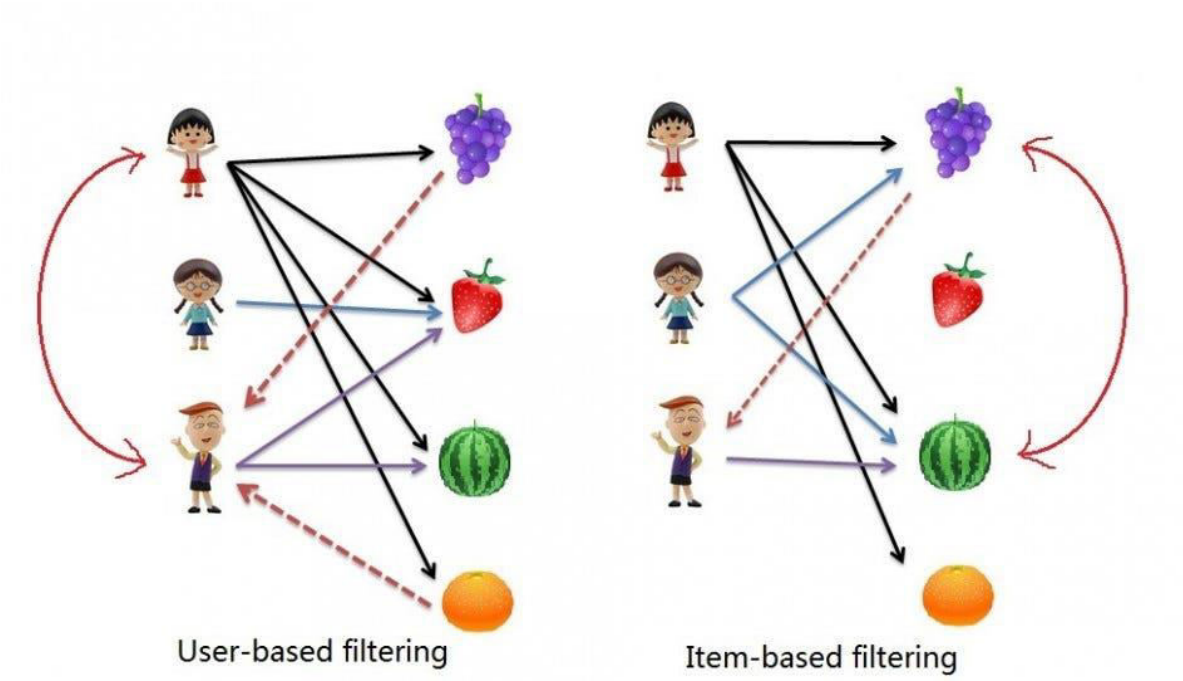
Example: If a user has watched and liked "Inception," the system may recommend similar movies like "Interstellar" or "The Matrix" based on item similarity.



Differences Between User-Based and Item-Based Collaborative Filtering

Feature	User-Based CF	Item-Based CF
Focus	Finds similar users	Finds similar items

Feature	User-Based CF	Item-Based CF
Computation	Dynamic and user-dependent	More stable and precomputed
Works Well For	Small user bases with frequent interactions	Large catalogs with infrequent interactions
Example	Social media friend suggestions	Amazon's "Customers also bought" section



Applications of Collaborative Filtering

- Movie and TV show recommendations (Netflix, Hulu)
- E-commerce product suggestions (Amazon, eBay)
- Music and podcast recommendations (Spotify, Apple Music)
- Online learning course recommendations (Coursera, Udemy)

Challenges of Collaborative Filtering

1. **Cold Start Problem** – New users or items have little interaction data.
2. **Sparsity** – User-item interaction matrices are often sparse.
3. **Scalability** – Large-scale data requires efficient computation.
4. **Diversity** – Over-recommendation of popular items can reduce diversity.

Advantages of Item-Based Collaborative Filtering

1. **Scalability** – Precomputing item similarities makes this approach more efficient for large datasets.
2. **Stability** – Unlike user-based filtering, item similarities do not change frequently, making recommendations more consistent.
3. **Better for Sparse Data** – Works well even when users interact with only a few items, as long as item similarities exist.
4. **Effective for Large Catalogs** – Since it focuses on item relationships rather than users, it scales well in scenarios like e-commerce platforms.

When to Use Item-Based Collaborative Filtering

- When there is a **large number of users** and frequent changes in user behavior.
- When the **dataset is sparse**, meaning each user interacts with only a few items.

- When a platform has a **large inventory** (e.g., Amazon, Netflix) where users interact with different items but patterns can still be found.
- When recommendations need to be **stable over time**, ensuring that users see consistent suggestions based on item similarity.

```python
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample user-item rating matrix (NaN indicates unrated items)
data = {
    'Item1': [5, 4, np.nan, 3, 5],
    'Item2': [3, np.nan, 4, 2, 4],
    'Item3': [np.nan, 5, 4, np.nan, 3],
    'Item4': [4, np.nan, 5, 3, 2],
    'Item5': [np.nan, 4, 3, 5, 4]
}
rating_matrix = pd.DataFrame(data)

# Compute item-item similarity matrix
item_similarity = cosine_similarity(rating_matrix.T)

# Convert to DataFrame for easier interpretation
item_sim_df = pd.DataFrame(item_similarity, index=rating_matrix.columns, columns=rating_matrix.columns)

# Get the ratings given by a specific user (user index 2 in this case)
user_index = 2
user_ratings = rating_matrix.iloc[user_index]

# Compute weighted scores for each item
scores = item_sim_df.dot(user_ratings).div(item_sim_df.sum(axis=1))

# Filter out items already rated by the user
scores = scores[user_ratings.isna()]
```

```python
# Filter out items already rated by the user
scores = scores[user_ratings.isna()]

# Get top 3 recommendations
recommendations = scores.nlargest(3)

print("Recommended Items:")
print(recommendations)
```

Conclusion

Collaborative filtering is a powerful recommendation technique that enhances user experience by suggesting relevant content. While both user-based and item-based approaches have their advantages, hybrid models combining both techniques are often used to improve accuracy and overcome challenges.